디자인 패턴

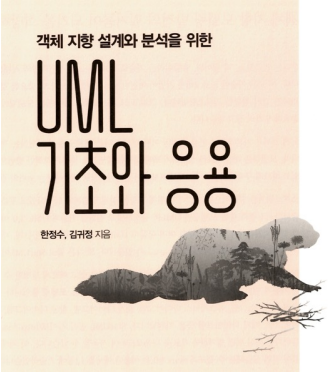
2026. 3. 17
미림 마이스터고

남궁성
seong.namkung@gmail.com

UML Unified Modeling Language

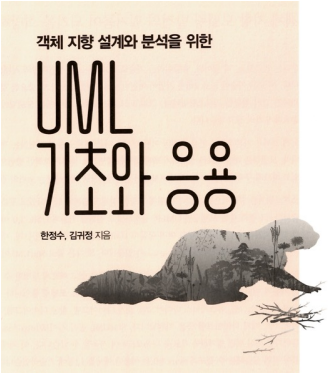
- 시스템 개발을 위한 시각적인 설계 표기를 제공
- UML은 개발 방법론이나 개발 프로세스, 프로그래밍 언어가 아니라 표준화된 모델링 언어
- 1990년대 다양한 객체지향 방법론이 존재

  OOD/Booch, OMT, OOAD, RR, GOOD, HOOD, OOSD, OOJSD 등
- 각 객체 지향 방법론은 독자적인 표기법과 절차를 정의. 호환성 문제 발생
- UML은 이런 문제점을 해결하기 위한 노력으로 탄생한 통한 모델링 언어



UML이란?

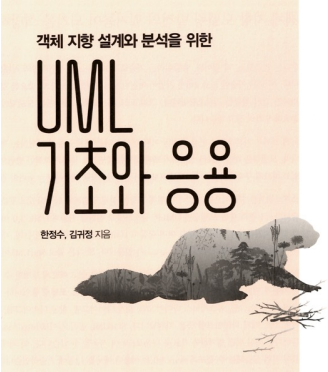
- UML은 시각화 언어다. - Visualization
- UML은 명세화 언어다. - Specification
- UML은 구축 언어다. - Construction
- UML은 문서화 언어다. - Documentation



UML의 다이어그램

- 정보 시스템을 다양한 시각화로 모델링가능하게 표준화된 다이어그램을 제공

  - 유스 케이스 다이어그램 Use-case Diagram
  - 클래스 다이어그램 Class Diagram
  - 순차 다이어그램 Sequence Diagram
  - 통신 다이어그램 Communication Diagram
  - 활동 다이어그램 Activity Diagram
  - 상태 다이어그램 State Diagram
  - 컴포넌트 다이어그램 Component Diagram
  - 배치 다이어그램 Deployment Diagram
  - 패키지 다이어그램 Package Diagram



객체 지향 모델링

- 모델링이란?

모델링은 개발하고자 하는 프로그램을 시각적으로 표현하는 것
의뢰자의 요구에 맞게 쉽게 수정해서 결과적으로 유지보수 기간을 줄여 생산성을 높일 수 있다.

- 객체 지향 모델링

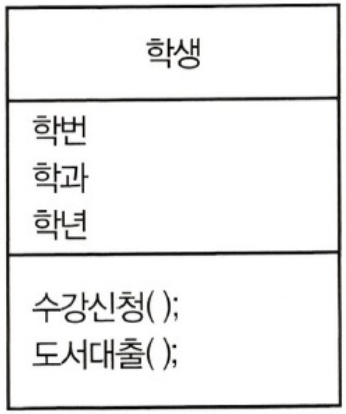
객체 지향 방법에서는 시스템을 객체의 관점으로 바라본다.
객체 지향 시스템을 가시화하고 명세화 & 문서화하는 것이 목적

객체, 클래스, 메시지

- 객체와 클래스

클래스 : 객체를 정의한 것

인스턴스 : 클래스로부터 생성된 객체



- 메시지

객체는 각각 독립적으로 존재. 다른 객체와 상호작용

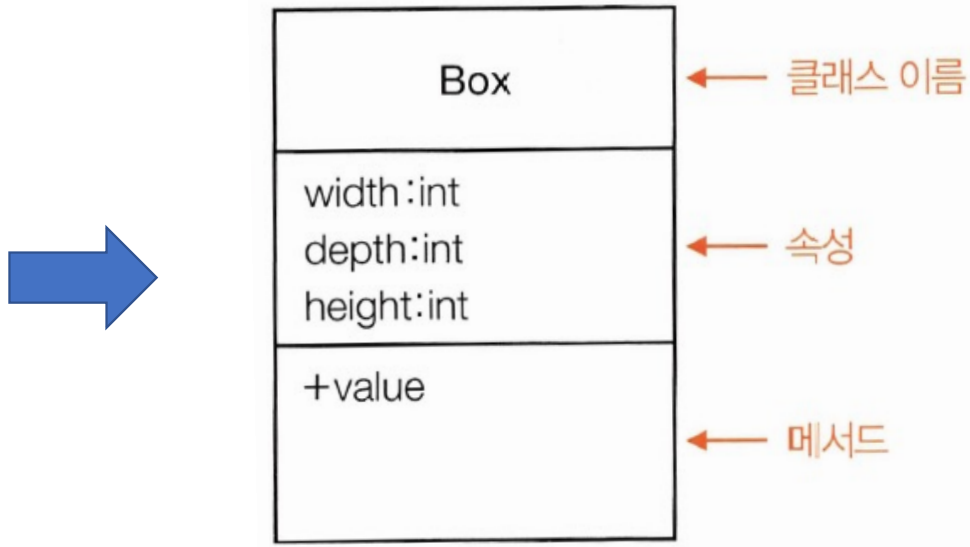
메시지는 객체간의 상호작용 수단

메시지는 한 객체(송신 객체)가 다른 객체(수신 객체)에게 작업을 요청하는 신호

클래스의 구성 요소

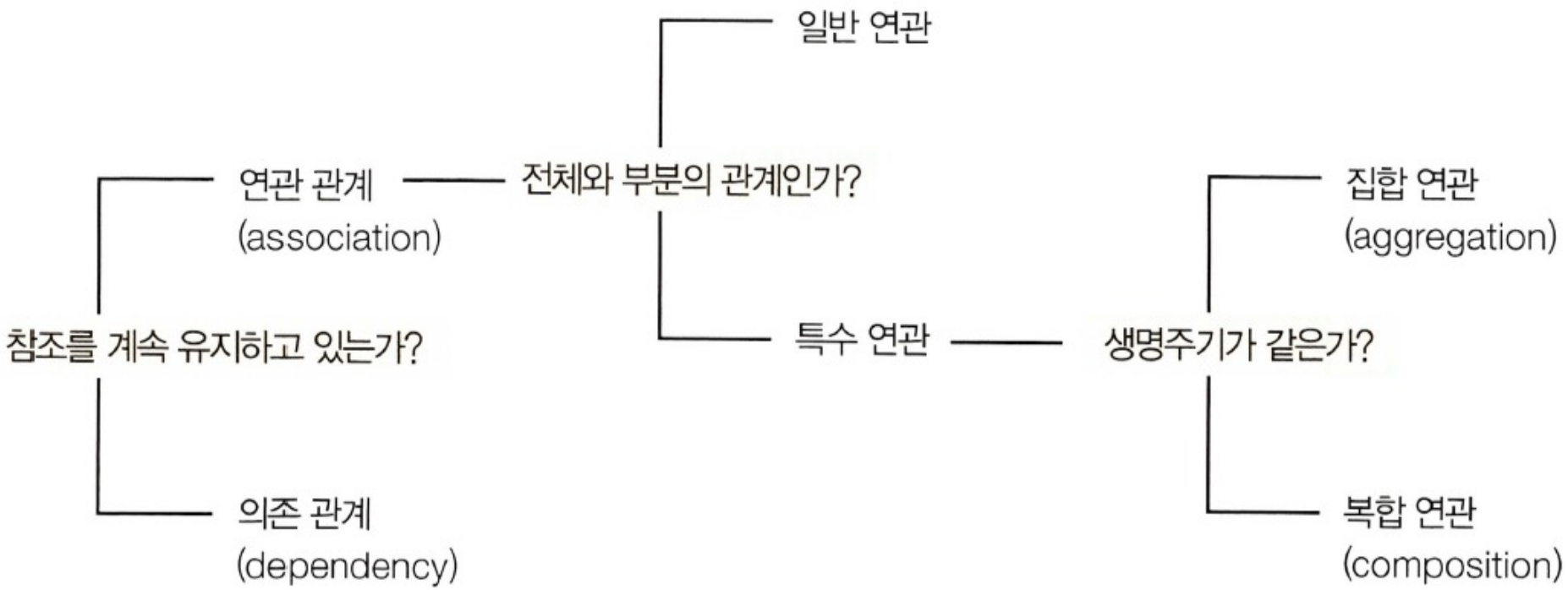
```
// 클래스
Box {
    // 속성
    private int width;
    private int depth;
    private int height;

    // 메서드
    void value() {
    }
}
```



종류	부호	접근 제어 범위
public	+	접근 제어가 없음
protected	#	같은 패키지 또는 다른 패키지의 자손
(default)	~	같은 패키지
private	-	같은 클래스

클래스 관계의 구분

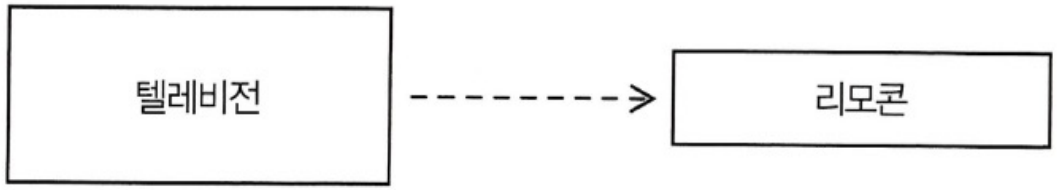

일반 연관
연관 관계
(association)
전체와 부분의 관계인가?
집합 연관
(aggregation)
참조를 계속 유지하고 있는가?
특수 연관
생명주기가 같은가?
의존 관계
(dependency)
복합 연관
(composition)

클래스와 객체의 관계 표현

1. 의존 관계(dependency)

- 한 클래스가 다른 클래스를 잠깐 사용하는 관계. 사용이 끝나면 관계도 끝
- 의존은 한 클래스를 다른 클래스의 오퍼레이션의 매개변수로 사용하는 경우에 주로 나타난다.

```
class A {
    void method(B b) {   // B를 잠깐 사용
        b.work();
    }
}
```



클래스와 객체의 관계 표현

2. 연관 관계(association)

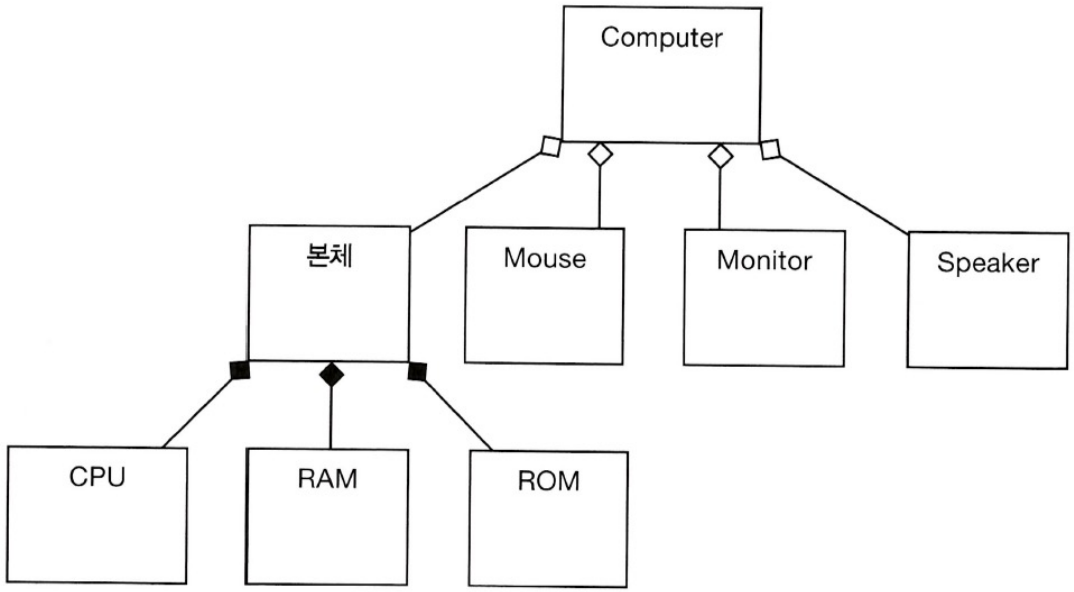
- 두 클래스의 관계로 지속적 유지. 한 클래스가 다른 클래스를 계속 알고 있음.
- 두 클래스가 연관 관계가 되면 한쪽 객체에서 다른 객체로 옮겨갈 수 있다.
- 연관의 양쪽 끝이 같은 클래스를 향하는 순환도 가능.(양방향 관계)

```
class A {
    B b;    // 계속 가지고 있음
}
```

클래스와 객체의 관계 표현

3. 집합 관계(aggregation)와 복합 관계(composite)

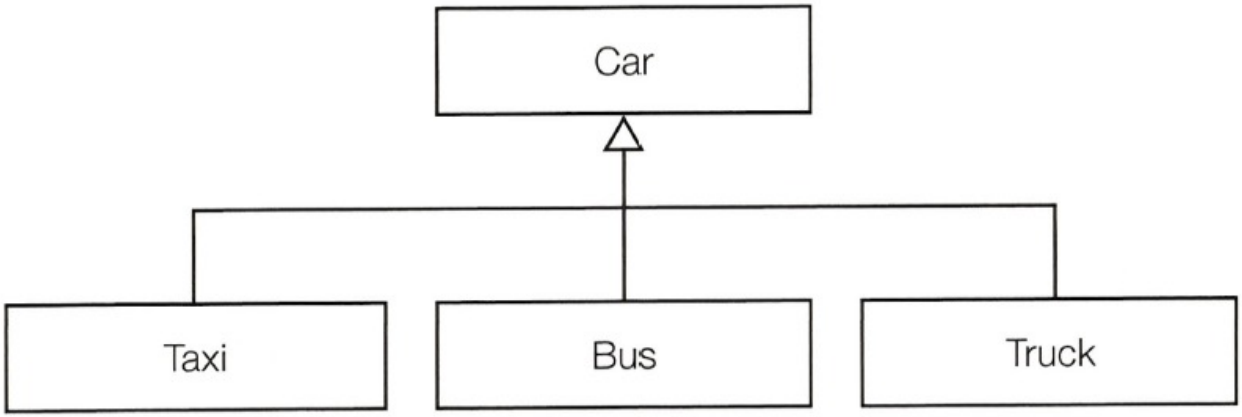
- 집합 관계 - 한 클래스가 더 작은 클래스들로 이루어지는 관계
- 복합 관계 - 집합 관계 중에서 더 강한 관계로 구성되는 경우



클래스와 객체의 관계 표현

4. 일반화 관계와 특수화 관계

- 일반화된 사물과 좀더 특수화된 사물 사이의 관계. 상속 관계
- 한 종류를 의미하며 kind-of관계 또는 is-a 관계라고도 한다.



디자인 패턴 목록(1/3)

1. Singleton – 인스턴스를 단 하나만 생성(5) p.102
2. Flyweight – 같은 것을 공유해서 절약(20) p.364
3. Prototype – 복사해서 인스턴스를 생성(6) p.112
4. Template Method – 하위 클래스에서 구체적으로 처리(3) p.74
5. Strategy – 알고리즘을 모두 바꿈(10) p.186
6. Bridge – 기능 계층과 구현 계층을 나눔(9) p.172
7. Composite – 그릇과 내용물을 동일시(11) p.204
8. Decorator – 장식물과 내용물을 동일시(12) p.218
9. Command – 명령을 클래스로 표현(22) p.394

디자인 패턴 목록(2/3)

10. State – 상태를 클래스로 표현(19) p.340
11. Proxy – 필요할 때 만들기(21) p.380
12. Observer – 상태 변화를 통보(17) p.308
13. Memento – 상태를 저장(18) p.322
14. Factory Method - 하위 클래스에서 인스턴스를 생성(4) p.88
15. Abstract Factory – 부품을 조립해서 제품을 만듦(8) p.144
16. Builder – 복잡한 인스턴스를 조립(7) p.128
17. Adapter – 사이에 끼워 재사용(2) p.58

디자인 패턴 목록(3/3)

18. Iterator – 처리를 반복(1) p.42
19. Visitor – 데이터 구조를 돌아다니면서 처리(13) p.240
20. Interpreter – 문법 규칙을 클래스로 표현(23) p.414
21. Chain of Responsibility – 책임을 떠넘기기(14) p.260
22. Facade – 단순한 창구 만들기(15) p.274
23. Mediator – 중재자를 통해 처리(16) p.288

객체지향개념 5가지 원칙 - SOLID

- SRP – 단일 책임의 원칙. 하나의 메서드는 하나의 기능만
- OCP – 개방 폐쇄 원칙(상속 Open, 변경 Close). 변경x 상속o
- LSP – 리스코프 치환 원칙. 조상을 자손으로 대체 가능해야
- ISP – 인터페이스 분리 원칙. 다른 목적의 인터페이스는 분리
- DIP – 의존성 역전 원칙. 구체화 의존x, 추상화 의존o
  클래스가 아닌, 인터페이스에 의존하라.

-1. 디자인 패턴 공부의 시작

생성자의 단점

1. 항상 새로운 객체를 생성 -> factory 메서드, Flyweight패턴
2. 매개변수가 많아지면 구분 어려움 -> Builder패턴

```
public class UserTest {
    public static void main(String[] args) {
//        User user = new User("kim", 20, "kim@test.com");
        User user = new User.Builder()
                .name("kim")
                .age(20)
                .email("kim@test.com")
                .build();
    }
}
```

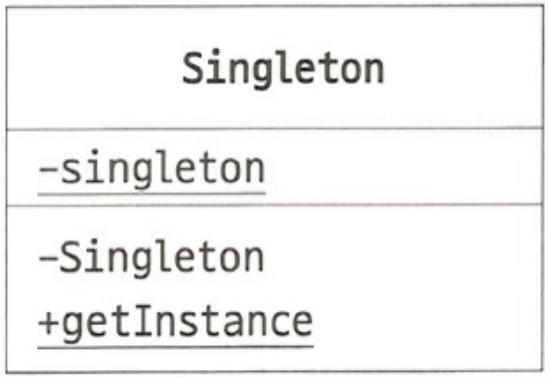
0. Factory method – 객체를 반환하는 메서드

```java
public class FactoryMethodTest {
    public static void main(String[] args) {
        Product p = new Product(); // 생성자
        Product p2 = Factory.getInstance(); // factory method
    }
}
```

```java
class Product {}  5 usages
class Factory {  1 usage
    static Product getInstance() {  1 usage
        return new Product();
    }
}
```

```java
class Product {}  6 usages
class Factory {  1 usage
    static Product p = new Product();  1 usage
    static Product getInstance() {  1 usage
        return p; // 미리 생성한 객체를 반환
    }
}
```

1. Singleton – 인스턴스를 단 하나만 생성(5) p.102



1. Singleton – 인스턴스를 단 하나만 생성(5) p.102

```java
public class Main {
    public static void main(String[] args) throws Exception {
        Singleton s  = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();
        System.out.println("s = " + s);
        System.out.println("s2 = " + s2);
    }
}
```

```java
public class Singleton {
    static Singleton s = new Singleton();

    private Singleton() {}

    static Singleton getInstance() {
        return s;
    }
}
```